

MediRun

시스템 아키텍처 구조도

현재 시연 구성 · ROS 2 데이터 흐름 · 안전 제어 · 추후 확장성

TurtleBot3 Burger · Raspberry Pi 4 · ROS 2 Humble · Nav2 · Arduino/ESP32 · Qt(C++) · PostgreSQL

PROJECT MEMBERS 도희정 · 최치언

2026.07.21 / REV.1.0

1. 전체 시스템 한눈에 보기

한 줄 요약
현장 버튼 입력을 로봇 내부의 ROS 2 미션으로 변환하고, Nav2로 목적지까지 이동한 뒤 AED 반출 여부에 따라 자동 복귀·도킹하며 모든 결과를 로컬 DB에 기록하는 구조다.

① 현장 입력·안내

SOS 버튼 3개
BTN_01 · BTN_02 · BTN_03

초음파 · 홀센서
낮은 장애물 / AED 적재·반출 감지

LED · 스피커
주행 상태와 안내 음성 출력

→ USB Serial

② TurtleBot3 / Raspberry Pi 4

arduino_bridge · sos_bridge
버튼 디바운스와 센서 입력을 ROS 2 이벤트로 변환

dispatcher · waypoint · mission
호출 검증 → 목적지 조회 → 미션 상태 관리

safety_manager
모든 속도 명령을 최종 검사하고 위험 시 0속도 출력

Nav2 · AMCL · TF · docking
자율주행, 위치 추정, 재계획, ArUco 정밀 파킹

로컬 PostgreSQL · event_logger
호출부터 복귀까지 미션·이벤트·센서 상태 저장

↔ ROS 2 Wi-Fi

③ 원격 운영 PC

Qt 대시보드
위치·미션·배터리·센서·연결 상태 표시

관리 명령
GUI 비상정지 · 미션 취소 · 홈 복귀

RViz2 · 개발 도구
지도·TF·경로 확인과 시험 지원

현재 시연의 시스템 경계
중앙 서버와 통합 DB는 사용하지 않는다. Wi-Fi가 끊겨도 주행·안전 판단·복귀·로컬 기록은 로봇 내부에서 계속 수행한다.

입력	판단	동작	기록·관제
버튼·센서	미션·안전 노드	Nav2·도킹·안내	로컬 DB·Qt

2. 로봇 내부 소프트웨어 구조

각 노드는 한 가지 책임을 갖는다. 호출 해석, 미션 제어, 안전 판단, 주행과 도킹을 분리해 오류 범위를 줄인다.

서비스 응용	**emergency_dispatcher** 버튼 ID·안전 조건 검증, 미션 요청 생성	**waypoint_manager** YAML 목적지·경로 ID 조회와 Pose 검증	**mission_manager** 상태 전이, Nav2 액션, 재시도·복귀 관리
안전·적재	**safety_manager** /cmd_vel 통과 여부와 GUI 비상정지 판단	**payload_monitor** 홀센서 기반 AED 적재·반출 확정	**network_monitor** 연결 상태 감지와 대시보드 재연결
주행·도킹	**Nav2 · AMCL** 경로 계획·제어와 map 기준 위치 추정	**TF · map_server** map→odom→base_link 좌표 관계 유지	**docking_manager** ArUco 정렬·접근·도킹 재시도
입출력·안내	**arduino_bridge** 버튼·초음파·홀센서·LED 시리얼 연결	**audio · led_manager** 상태별 음성·경광 LED 출력	**telemetry_bridge** 관제용 위치·배터리·센서 상태 정리
데이터·실행	**event_logger** 미션 ID로 이벤트를 연결해 저장	**PostgreSQL** ROBOT_SESSION · SYSTEM_EVENT 등 로컬 기록	**PM2 / systemd** 전원 인가 시 자동 시작과 장애 시 재시작

2.1 핵심 책임 분리

dispatcher	mission_manager	safety_manager	docking_manager	Qt 대시보드
호출을 해석하지만 Nav2를 직접 제어하지 않는다.	자율주행 목표 생성·취소·재시도를 단독 관리한다.	모든 속도 명령보다 높은 우선순위로 동작한다.	도킹 최종 접근 구간만 직접 제어한다.	관제 화면이며 로봇 내부 안전 루프를 대체하지 않는다.

> **쉽게 이해하면:** dispatcher는 접수 담당, mission_manager는 운행 관리자, Nav2는 길찾기 기사, safety_manager는 최종 안전 감독자다.

3. 버튼 호출부터 자동 복귀까지

정상 흐름
입력 이벤트와 주행 제어를 바로 연결하지 않고, 검증·목적지 조회·미션 상태 관리를 차례로 거쳐 안전하게 실행한다.

1. **SOS 버튼** — BTN_01~03 중 하나를 누른다.
2. **입력 변환** — Arduino/ESP32가 300ms 디바운스 후 ID를 전송한다.
3. **호출 검증** — dispatcher가 배터리·AED·AMCL·Nav2 상태를 확인한다.
4. **목적지 조회** — waypoint_manager가 버튼에 맞는 목적지 Pose를 찾는다.
5. **미션 생성** — mission_manager가 mission_id와 Nav2 목표를 생성한다.
6. **자율주행** — Nav2가 경로 계획·장애물 회피·재계획을 수행한다.
7. **도착·대기** — 목적지 10cm 이내 정지 후 반출을 기다린다.
8. **반출 판단** — 홀센서 이탈 0.5초 유지 또는 3분 미반출을 판단한다.
9. **복귀·도킹** — 홈 이동 후 ArUco로 최종 파킹한다.
10. **완료·기록** — IDLE 전환, Qt 갱신, 로컬 DB에 전체 결과 저장

3.1 현재 시연 버튼 매핑

BTN_01	BTN_02	BTN_03
DEMO_WP_01	DEMO_WP_02	DEMO_WP_03

3.2 핵심 ROS 2 인터페이스

이름	방향	쉬운 의미
/medirun/sos/event	bridge → dispatcher	어느 버튼이 눌렸는지 알림
/medirun/mission/request	dispatcher → manager	검증된 배송 미션 요청
/navigate_to_pose	manager → Nav2	지도상의 목표 좌표로 이동
/medirun/payload/state	hall → payload monitor	AED 적재·반출 상태
/medirun/mission/status	manager → Qt/logger	현재 미션 단계와 결과

4. 상태 전이와 안전 제어

미션 상태를 명확히 나누면 현재 동작과 실패 위치를 Qt 화면과 DB에서 동일하게 추적할 수 있다.



주행 실패
NAVIGATING → RETRY_WAIT → 최대 2회 재시도 → FAILED

도킹 실패
0.2m 후퇴·재탐색 → 최대 3회 → home_wait 정지

위치 유실
AMCL 무효 시 현재 위치 정지, 신규 목표 차단

4.1 모든 속도는 안전 관리자를 통과

속도 명령 입력
Nav2 주행 명령
docking_manager 최종 접근
관리자 수동 제어

→

safety_manager
위험 조건 확인
통과 또는 0속도 출력

→

TurtleBot3 Base
OpenCR · 모터
검증된 /cmd_vel만 실행

4.2 주요 안전 조건

조건	판단	처리
GUI 비상정지	/medirun/emergency_stop 활성	즉시 0속도, 안전 조건 충족 전 이동 금지
낮은 장애물	초음파 15cm 이내	감속·정지 후 Nav2 재계획
배터리 부족	30% 미만 신규 미션 거부 / 20% 미만	활성 미션보다 홈 복귀 우선
AMCL·TF 오류	map 기준 Pose 무효	신규 목표 차단, 현재 위치 정지
AED 미적재	출동 전 홀센서 HALL:0	미션 거부, Qt에 적재 요청 표시

판단 우선순위 안전정지 → 배터리 복귀 → 활성 배송 → 대기 미션 → 복귀·도킹 → IDLE

5. 현재 시연과 추후 확장성

핵심 원칙
버튼은 호출 구역을 나타내고, 웨이포인트는 로봇의 목적지·경유점·홈·도킹 위치를 나타낸다. 두 개수는 서로 같을 필요가 없다.



5.1 무엇을 언제 추가할까?

버튼 추가
새로운 호출 장소가 생길 때 추가한다.
호출 구역 기준

웨이포인트 추가
경로가 길거나 회전·분기·복구 기준점이 필요할 때 추가한다.
주행 경로 기준

통신 보완
Wi-Fi 음영은 AP·메시·네트워크 설계로 해결한다. 웨이포인트는 통신 중계점이 아니다.
네트워크 기준

5.2 단계별 확장 로드맵

단계	추가 구성	아키텍처 변화
현재 시연	버튼 3개, 목적지 3개, 로봇 1대	USB Serial 입력, 온보드 ROS 2, 로컬 PostgreSQL
구역 확장	분산 ESP32 Wi-Fi 버튼	button_id·zone·상태 heartbeat 관리 추가
경로 확장	초기 확장안 11개와 route_id	목적지 단일 Pose에서 경로 시퀀스 선택으로 확장
운영 확장	다중 로봇·선택적 중앙 관제	로봇 할당, 미션 큐, 통합 이력은 별도 확장 계층으로 분리

최종 구조 요약
버튼 → 호출 목적지 선택, 경유 웨이포인트 → 안정적인 주행 경로 구성, Nav2 → 실제 이동, safety_manager → 최종 안전 통제, 로컬 DB → 네트워크와 무관한 이력 보존.